Step III: Crashing and Recovering

What I Built

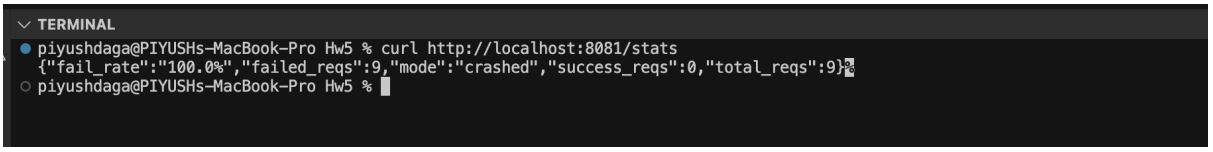
I extended my Product API (Go/Gin) to depend on a downstream Inventory Service for stock information. The Inventory Service can be switched between healthy, degraded, and crashed modes to simulate real-world failures.

When the inventory service goes down, the product API has to handle it and without any protection, it doesn't handle it well at all.

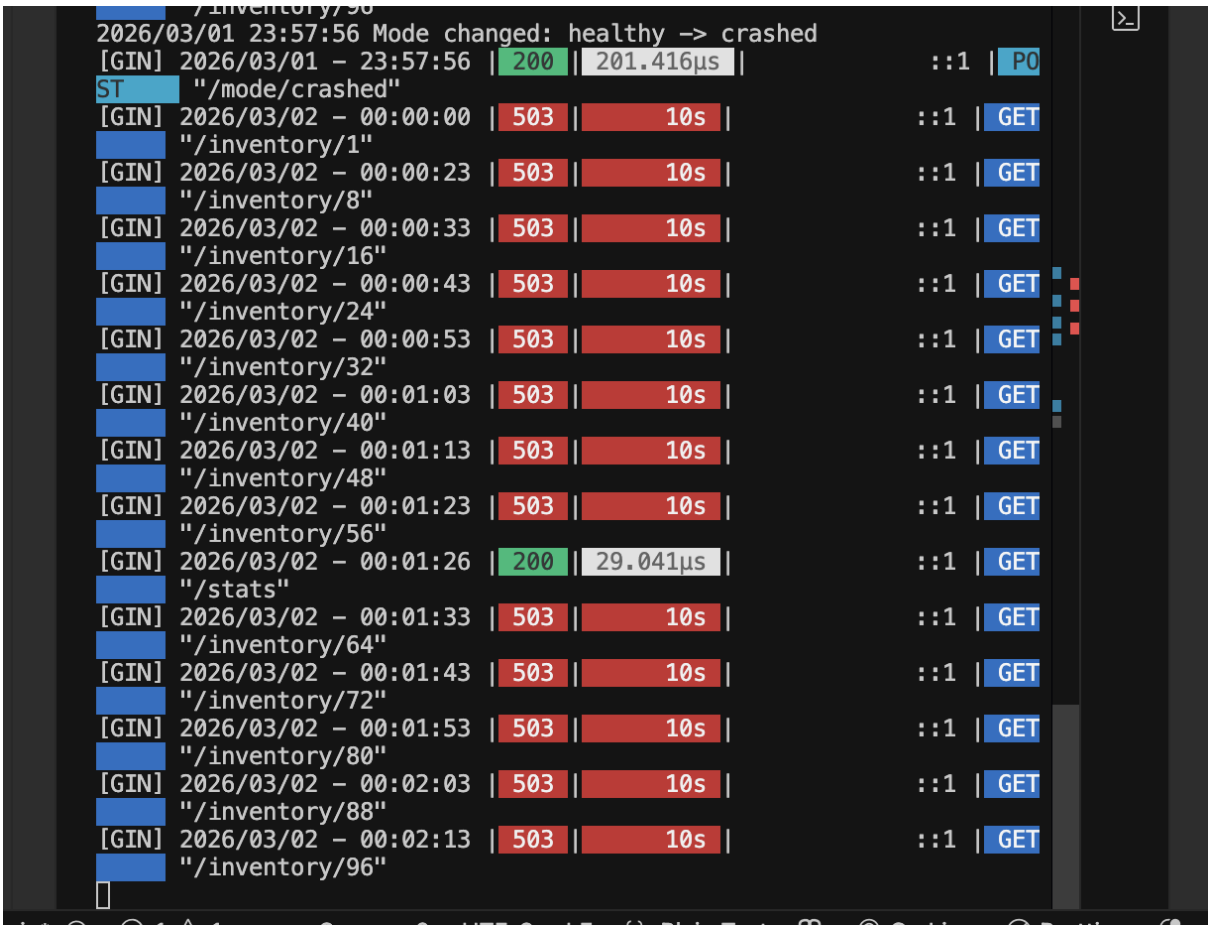
The Problem (No Resilience)

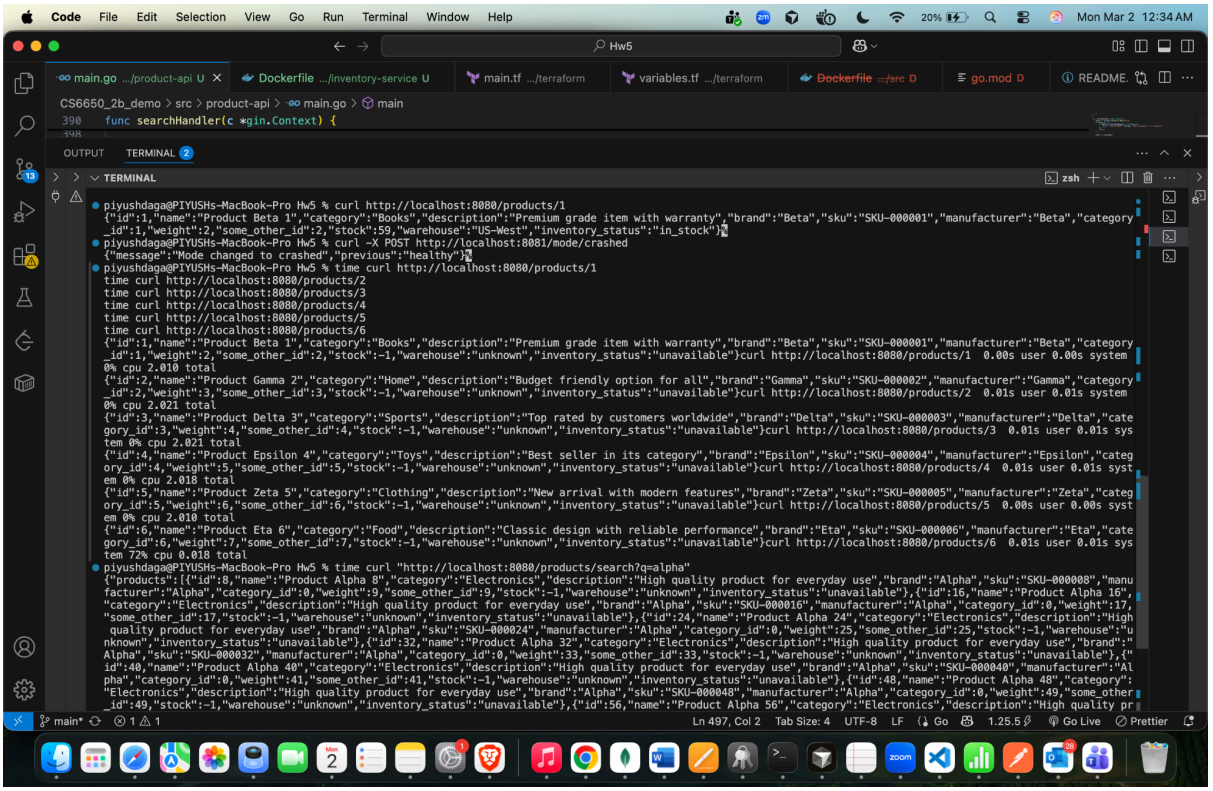
My initial implementation calls the inventory service directly with no timeout, no circuit breaker, nothing. When inventory crashes:

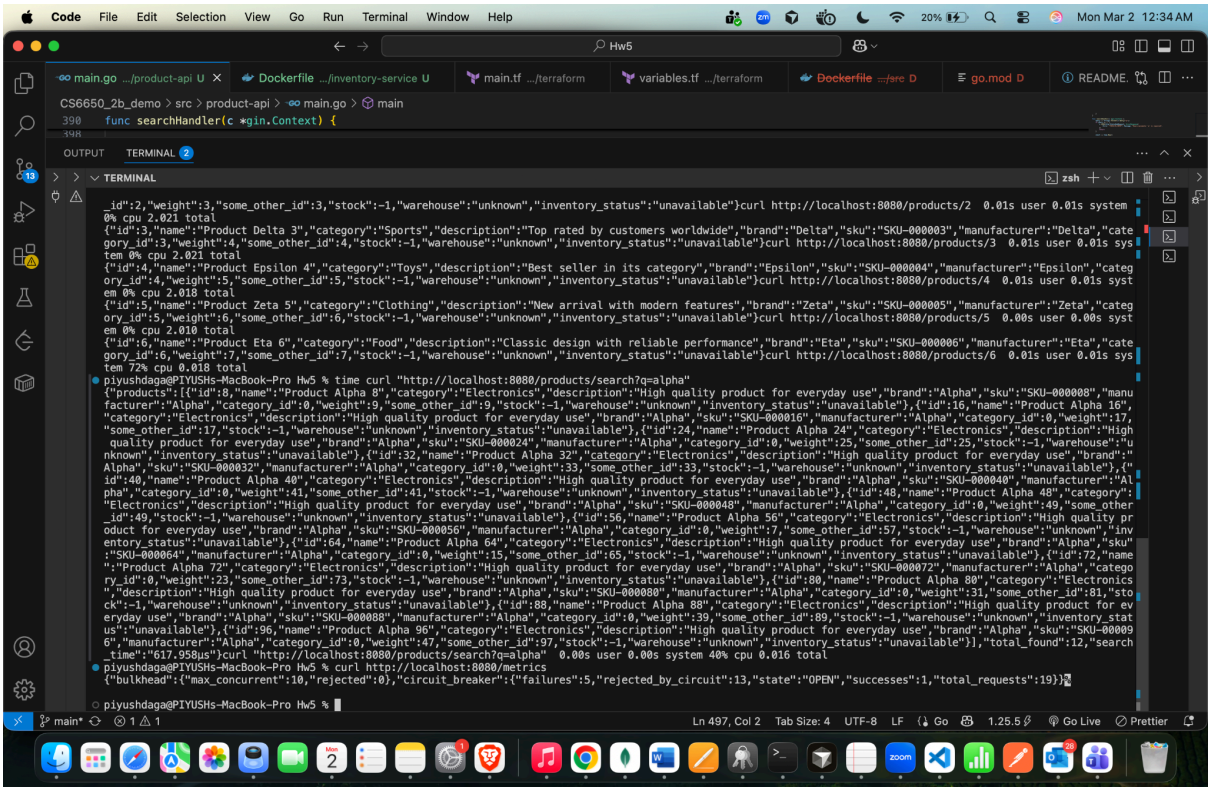
- **Single product request**: hangs for **~10 seconds** waiting for inventory to respond
- **Search (12 results)**: takes **2 minutes** because each result calls inventory sequentially (12 × 10s)
- Every incoming request blocks a goroutine waiting on a dead service
- The entire API becomes unusable

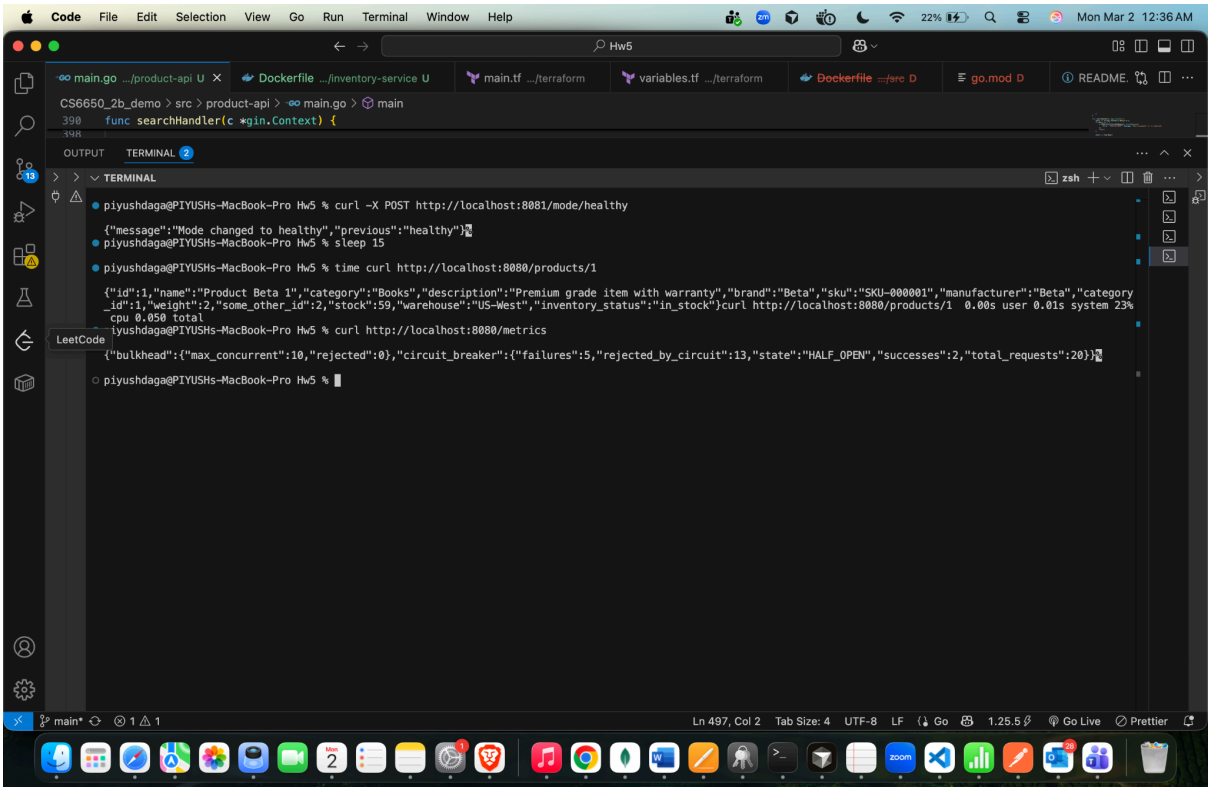

TERMINAL
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl http://localhost:8081/stats
{"fail_rate":"100.0%","failed_reqs":9,"mode":"crashed","success_reqs":0,"total_reqs":9}%
piyushdaga@PIYUSHs-MacBook-Pro Hw5 %


Hw5
OUTPUT
TERMINAL 2
TERMINAL
zsh
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl http://localhost:8081/stats
{"fail_rate":"0.0%","failed_reqs":0,"mode":"healthy","success_reqs":1,"total_reqs":1}%
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl "http://localhost:8080/products/search?q=alpha"
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl -X POST http://localhost:8081/mode/crashed
{"message":"Mode changed to crashed","previous":"healthy"}%
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % time curl http://localhost:8080/products/1
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % time curl "http://localhost:8080/products/search?q=alpha"
main*
Ln 16, Col 1
Tab Size: 4
UTF-8
LF
Go
1.25.5
Go Live
Prettier


2026/03/01 23:57:56 Mode changed: healthy -> crashed
[GIN] 2026/03/01 - 23:57:56 | 200 | 201.416µs | ::1 | POST "/mode/crashed"
[GIN] 2026/03/02 - 00:00:00 | 503 | 10s | ::1 | GET "/inventory/1"
[GIN] 2026/03/02 - 00:00:23 | 503 | 10s | ::1 | GET "/inventory/8"
[GIN] 2026/03/02 - 00:00:33 | 503 | 10s | ::1 | GET "/inventory/16"
[GIN] 2026/03/02 - 00:00:43 | 503 | 10s | ::1 | GET "/inventory/24"
[GIN] 2026/03/02 - 00:00:53 | 503 | 10s | ::1 | GET "/inventory/32"
[GIN] 2026/03/02 - 00:01:03 | 503 | 10s | ::1 | GET "/inventory/40"
[GIN] 2026/03/02 - 00:01:13 | 503 | 10s | ::1 | GET "/inventory/48"
[GIN] 2026/03/02 - 00:01:23 | 503 | 10s | ::1 | GET "/inventory/56"
[GIN] 2026/03/02 - 00:01:26 | 200 | 29.041µs | ::1 | GET "/stats"
[GIN] 2026/03/02 - 00:01:33 | 503 | 10s | ::1 | GET "/inventory/64"
[GIN] 2026/03/02 - 00:01:43 | 503 | 10s | ::1 | GET "/inventory/72"
[GIN] 2026/03/02 - 00:01:53 | 503 | 10s | ::1 | GET "/inventory/80"
[GIN] 2026/03/02 - 00:02:03 | 503 | 10s | ::1 | GET "/inventory/88"
[GIN] 2026/03/02 - 00:02:13 | 503 | 10s | ::1 | GET "/inventory/96"


Code File Edit Selection View Go Run Terminal Window Help
Mon Mar 2 12:34 AM
Hw5
main.go .../product-api U
Dockerfile .../inventory-service U
main.tf .../terraform
variables.tf .../terraform
Dockerfile .../src D
go.mod D
README.
CS6650_2b_demo > src > product-api > main.go > main
func searchHandler(c *gin.Context) {
OUTPUT
TERMINAL 2
TERMINAL
zsh
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl http://localhost:8080/products/1
{"id":1,"name":"Product Beta 1","category":"Books","description":"Premium grade item with warranty","brand":"Beta","sku":"SKU-000001","manufacturer":"Beta","category_id":1,"weight":2,"some_other_id":2,"stock":59,"warehouse":"US-West","inventory_status":"in_stock"}
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl -X POST http://localhost:8081/mode/crashed
{"message":"Mode changed to crashed","previous":"healthy"}
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % time curl http://localhost:8080/products/1
time curl http://localhost:8080/products/2
time curl http://localhost:8080/products/3
time curl http://localhost:8080/products/4
time curl http://localhost:8080/products/5
time curl http://localhost:8080/products/6
{"id":1,"name":"Product Beta 1","category":"Books","description":"Premium grade item with warranty","brand":"Beta","sku":"SKU-000001","manufacturer":"Beta","category_id":1,"weight":2,"some_other_id":2,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/1 0.00s user 0.00s system 0% cpu 2.010 total
{"id":2,"name":"Product Gamma 2","category":"Home","description":"Budget friendly option for all","brand":"Gamma","sku":"SKU-000002","manufacturer":"Gamma","category_id":2,"weight":3,"some_other_id":3,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/2 0.01s user 0.01s system 0% cpu 2.021 total
{"id":3,"name":"Product Delta 3","category":"Sports","description":"Top rated by customers worldwide","brand":"Delta","sku":"SKU-000003","manufacturer":"Delta","category_id":3,"weight":4,"some_other_id":4,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/3 0.01s user 0.01s system 0% cpu 2.021 total
{"id":4,"name":"Product Epsilon 4","category":"Toys","description":"Best seller in its category","brand":"Epsilon","sku":"SKU-000004","manufacturer":"Epsilon","category_id":4,"weight":5,"some_other_id":5,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/4 0.01s user 0.01s system 0% cpu 2.018 total
{"id":5,"name":"Product Zeta 5","category":"Clothing","description":"New arrival with modern features","brand":"Zeta","sku":"SKU-000005","manufacturer":"Zeta","category_id":5,"weight":6,"some_other_id":6,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/5 0.00s user 0.00s system 0% cpu 2.010 total
{"id":6,"name":"Product Eta 6","category":"Food","description":"Classic design with reliable performance","brand":"Eta","sku":"SKU-000006","manufacturer":"Eta","category_id":6,"weight":7,"some_other_id":7,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"}curl http://localhost:8080/products/6 0.01s user 0.01s system 72% cpu 0.018 total
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % time curl "http://localhost:8080/products/search?q=alpha"
{"products":[{"id":8,"name":"Product Alpha 8","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000008","manufacturer":"Alpha","category_id":0,"weight":9,"some_other_id":9,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":16,"name":"Product Alpha 16","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000016","manufacturer":"Alpha","category_id":0,"weight":17,"some_other_id":17,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":24,"name":"Product Alpha 24","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000024","manufacturer":"Alpha","category_id":0,"weight":25,"some_other_id":25,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":32,"name":"Product Alpha 32","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000032","manufacturer":"Alpha","category_id":0,"weight":33,"some_other_id":33,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":40,"name":"Product Alpha 40","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000040","manufacturer":"Alpha","category_id":0,"weight":41,"some_other_id":41,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":48,"name":"Product Alpha 48","category":"Electronics","description":"High quality product for everyday use","brand":"Alpha","sku":"SKU-000048","manufacturer":"Alpha","category_id":0,"weight":49,"some_other_id":49,"stock":-1,"warehouse":"unknown","inventory_status":"unavailable"},{"id":56,"name":"Product Alpha 56","category":"Electronics","description":"High quality pr
main*
Ln 497, Col 2 Tab Size: 4 UTF-8 LF Go 1.25.5 Go Live Prettier






```
piyushdaga@PIYUSHs-MacBook-Pro Hw5 % curl http://localhost:8080/products/2
curl http://localhost:8080/products/3
curl http://localhost:8080/metrics
{"id":2,"name":"Product Gamma 2","category":"Home","description":"Budget friendly option for all","brand":"Gamma","sku":"SKU-000002","manufacturer":"Gamma","category_id":2,"weight":3,"some_other_id":3,"stock":76,"warehouse":"EU-Central","inventory_status":"in_stock"}{"id":3,"name":"Product Delta 3","category":"Sports","description":"Top rated by customers worldwide","brand":"Delta","sku":"SKU-000003","manufacturer":"Delta","category_id":3,"weight":4,"some_other_id":4,"stock":93,"warehouse":"US-East","inventory_status":"in_stock"}{"bulkhead":{"max_concurrent":10,"rejected":0},"circuit_breaker":{"failures":5,"rejected_by_circuit":13,"state":"CLOSED","successes":4,"total_requests":22}}
piyushdaga@PIYUSHs-MacBook-Pro Hw5 %
```

The Fix — Three Resilience Patterns

I implemented three patterns from Sam Newman's approach, all hand-written in Go (no external libraries):

1. Fail Fast (Timeout)

Instead of Go's default HTTP client with no timeout, I set a **2-second timeout**. If inventory doesn't respond in 2s, we give up immediately instead of hanging for 10+ seconds.

```
var httpClient = &http.Client{
    Timeout: 2 * time.Second,
}
```

2. Circuit Breaker

Tracks consecutive failures. After **5 failures**, the circuit "opens" and all subsequent requests are **rejected instantly** we don't even try calling inventory. After **15 seconds**, it enters "half-open" state and tests one request. If it succeeds 3 times, the circuit closes and we're back to normal.

States: CLOSED (normal) → OPEN (rejecting) → HALF_OPEN (testing) → CLOSED (recovered)

3. Bulkhead

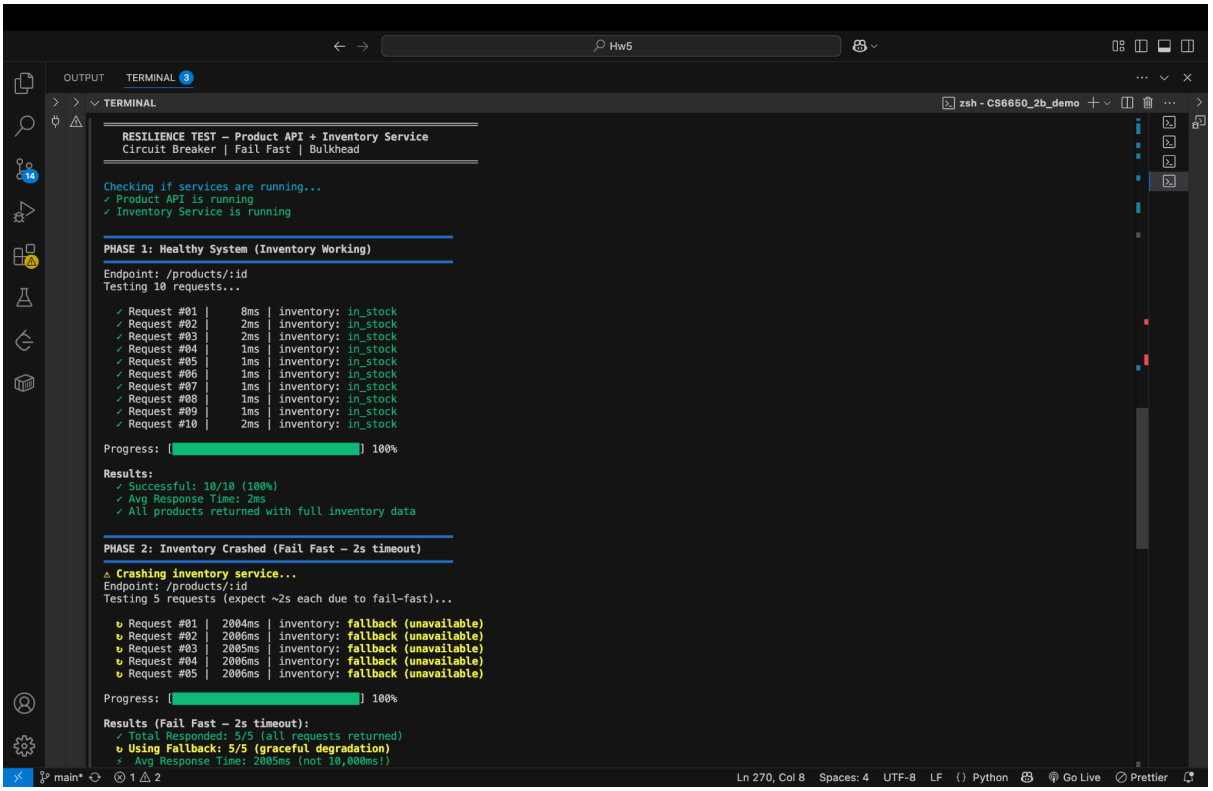
Limits concurrent inventory calls to **10**. If 100 users hit the API at once, only 10 will call inventory the rest get an instant fallback. This prevents a slow service from consuming all resources.

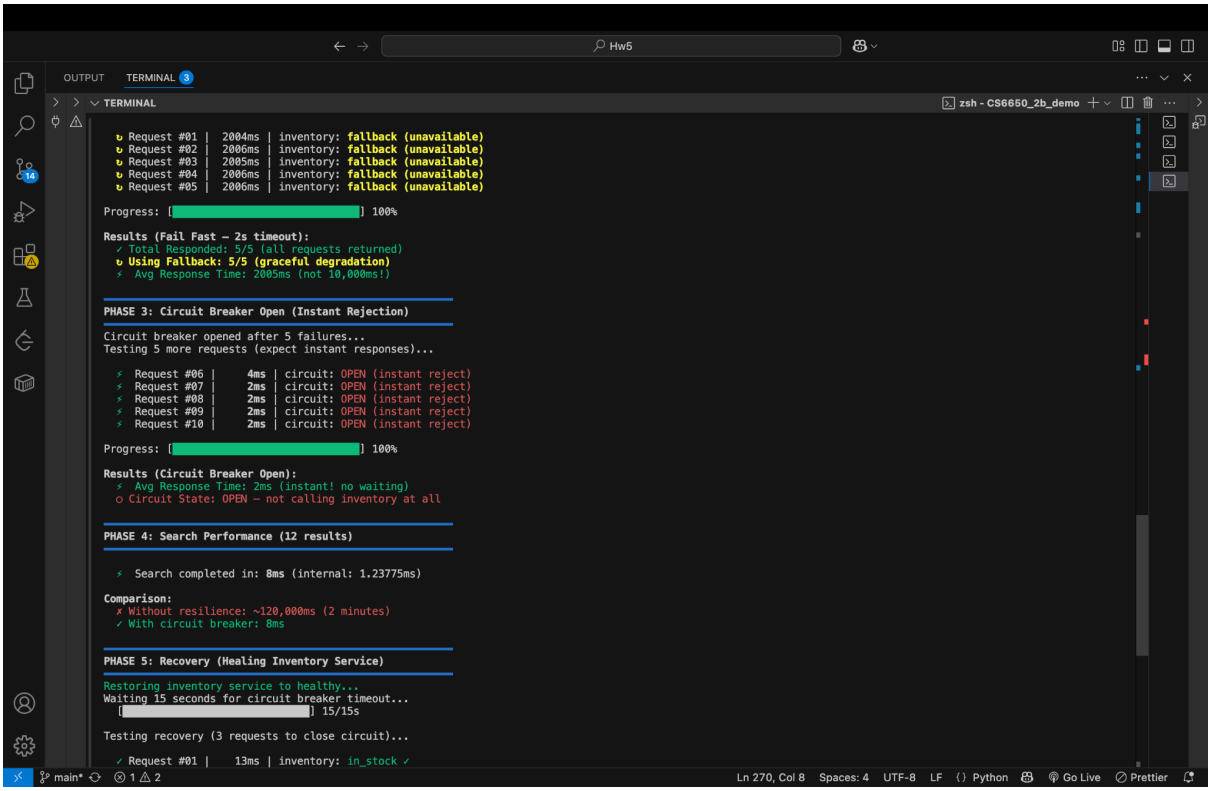
```
func (b *Bulkhead) Acquire() bool {
    select {
    case b.sem <- struct{}{}:
        return true
    default:
        return false  // reject immediately
    }
}
```

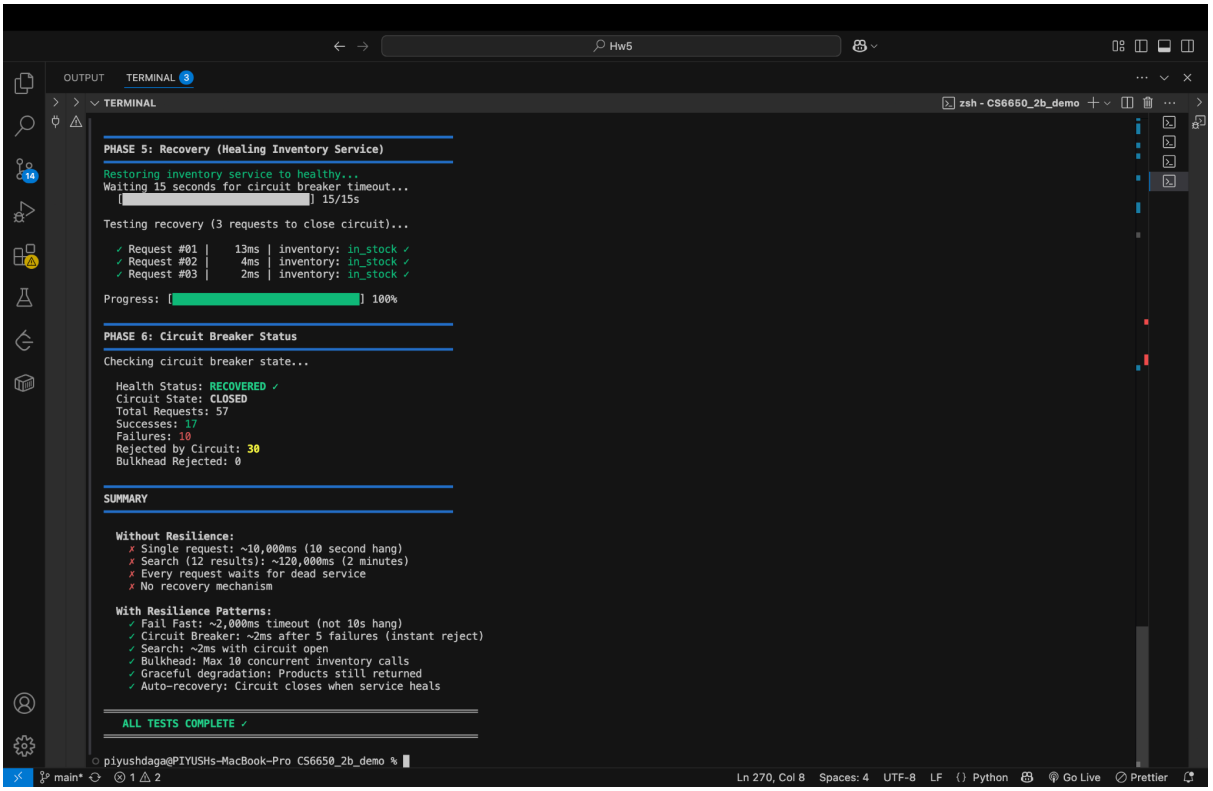
Graceful Degradation

When inventory is unavailable, products still return just without stock info (`inventory_status: "unavailable"`). The API never fully breaks.

Results


Hw5
OUTPUT
TERMINAL 3
TERMINAL
zsh - CS6650_2b_demo
RESILIENCE TEST — Product API + Inventory Service
Circuit Breaker | Fail Fast | Bulkhead
Checking if services are running...
✓ Product API is running
✓ Inventory Service is running
PHASE 1: Healthy System (Inventory Working)
Endpoint: /products/:id
Testing 10 requests...
✓ Request #01 | 8ms | inventory: in_stock
✓ Request #02 | 2ms | inventory: in_stock
✓ Request #03 | 2ms | inventory: in_stock
✓ Request #04 | 1ms | inventory: in_stock
✓ Request #05 | 1ms | inventory: in_stock
✓ Request #06 | 1ms | inventory: in_stock
✓ Request #07 | 1ms | inventory: in_stock
✓ Request #08 | 1ms | inventory: in_stock
✓ Request #09 | 1ms | inventory: in_stock
✓ Request #10 | 2ms | inventory: in_stock
Progress: [ ] 100%
Results:
✓ Successful: 10/10 (100%)
✓ Avg Response Time: 2ms
✓ All products returned with full inventory data
PHASE 2: Inventory Crashed (Fail Fast — 2s timeout)
⚠ Crashing inventory service...
Endpoint: /products/:id
Testing 5 requests (expect ~2s each due to fail-fast)...
↻ Request #01 | 2004ms | inventory: fallback (unavailable)
↻ Request #02 | 2006ms | inventory: fallback (unavailable)
↻ Request #03 | 2005ms | inventory: fallback (unavailable)
↻ Request #04 | 2006ms | inventory: fallback (unavailable)
↻ Request #05 | 2006ms | inventory: fallback (unavailable)
Progress: [ ] 100%
Results (Fail Fast — 2s timeout):
✓ Total Responded: 5/5 (all requests returned)
↻ Using Fallback: 5/5 (graceful degradation)
⚡ Avg Response Time: 2005ms (not 10,000ms!)
main*
1 2
Ln 270, Col 8
Spaces: 4
UTF-8
LF
Python
Go Live
Prettier


Hw5
OUTPUT
TERMINAL 3
TERMINAL
zsh - CS6650_2b_demo
↻ Request #01 | 2004ms | inventory: fallback (unavailable)
↻ Request #02 | 2006ms | inventory: fallback (unavailable)
↻ Request #03 | 2005ms | inventory: fallback (unavailable)
↻ Request #04 | 2006ms | inventory: fallback (unavailable)
↻ Request #05 | 2006ms | inventory: fallback (unavailable)
Progress: [ ] 100%
Results (Fail Fast — 2s timeout):
✓ Total Responded: 5/5 (all requests returned)
↻ Using Fallback: 5/5 (graceful degradation)
⚡ Avg Response Time: 2005ms (not 10,000ms!)
PHASE 3: Circuit Breaker Open (Instant Rejection)
Circuit breaker opened after 5 failures...
Testing 5 more requests (expect instant responses)...
⚡ Request #06 | 4ms | circuit: OPEN (instant reject)
⚡ Request #07 | 2ms | circuit: OPEN (instant reject)
⚡ Request #08 | 2ms | circuit: OPEN (instant reject)
⚡ Request #09 | 2ms | circuit: OPEN (instant reject)
⚡ Request #10 | 2ms | circuit: OPEN (instant reject)
Progress: [ ] 100%
Results (Circuit Breaker Open):
⚡ Avg Response Time: 2ms (instant! no waiting)
○ Circuit State: OPEN — not calling inventory at all
PHASE 4: Search Performance (12 results)
⚡ Search completed in: 8ms (internal: 1.23775ms)
Comparison:
✗ Without resilience: ~120,000ms (2 minutes)
✓ With circuit breaker: 8ms
PHASE 5: Recovery (Healing Inventory Service)
Restoring inventory service to healthy...
Waiting 15 seconds for circuit breaker timeout...
[ ] 15/15s
Testing recovery (3 requests to close circuit)...
✓ Request #01 | 13ms | inventory: in_stock ✓
main*
1 2
Ln 270, Col 8
Spaces: 4
UTF-8
LF
Python
Go Live
Prettier



Summary of Results

Metric	Without Resilience	With Resilience
Single product request	~10,000ms (10s hang)	~2,000ms → ~2ms after circuit opens
Search (12 results)	~120,000ms (2 min)	~8ms
Recovery	None	Automatic (15s timeout, then heals)

Test Script Output Highlights

- **Phase 1** — Healthy: 10/10 requests, avg 2ms, all with stock data
- **Phase 2** — Crashed + Fail Fast: 5/5 responded in ~2005ms (not 10s), graceful fallback
- **Phase 3** — Circuit Breaker Open: ~2ms per request (instant rejection, not calling inventory)
- **Phase 4** — Search: 8ms with circuit open vs 2 minutes without resilience
- **Phase 5** — Recovery: inventory healed, circuit moved HALF_OPEN → CLOSED, full data returned
- **Phase 6** — Final metrics: Circuit CLOSED, 17 successes, 10 failures, 30 rejected by circuit

The worst failures aren't when a service is completely dead (you get an instant connection refused). The worst is when a service is alive but unresponsive it accepts connections but never responds. That's what eats up all your threads and causes cascading failures. The

circuit breaker pattern specifically addresses this by stopping the bleeding after a threshold of failures.